⇒ What are subarrays ?

Contiguous part of an array.

⇒ How many subarrays ?

$arr[N] \rightarrow \frac{N(N+1)}{2}$

ex ⇒ [3 6 5 7 1 2 10]

	subarr	subsequences
⇒ 5 1 2	✗	✓
3 2 10	✗	✓
5 10	✗	✓

Subsequence : any sequence that can be generated by deleting 0 or more element from array

↓

sequence (order matters. [order should be same as array, not required to be contiguous]

arr ⇒ [3 6 5 7 1 2 10] ⇒ [7 2 10] ✓

✗ ✗ ✗ ✓ ✗ ✓ ✓

✗ ✓ ✗ ✗ ✓ ✗ ✗ ⇒ [6 1] ✓

✗ ✗ ✗ ✗ ✗ ✗ ✗ ⇒ [] ✓ [empty subseq]

✓ ✓ ✓ ✓ ✓ ✓ ✓ ⇒ [3 6 5 7 1 2 10] ✓

Quiz

arr ⇒ [1, 2, 3, 4, 5]

1) 1 2 3 4 5 ✓

2) 4 ✓

3) 2 3 5 ✓

4) 5 4 3 ← not a subseq. order is not followed.

ex ⇒ A = [-1 4 3 9]

	subseq	subarr
[-1 4]	✓	✓
[4 3 9]	✓	✓
[-1 3 9]	✓	✗
[3]	✓	✓
[9 3]	✗	✗

Note:- all subarr are subsequences but vice versa is not true.

⇒ A ⇒ [4 -1 2] ⟶ sort ⟶ [-1 2 4]

→ max

Subseq ⇒

[ ]	✓	0	✓ [ ]	0
[4]	✓	4	✓ [4]	4
[-1]	✓	-1	✓ [-1]	-1
[2]	✓	2	✓ [2]	2
[4 -1]	✗	4	✗ [-1 4]	4
[4 2]	✗	4	✓ [-1 2]	2
[-1 2]	✓	2	✗ [2 4]	4
[4 -1 2]	✗	4	[-1 2 4] ✗	4

* Subsequence might change after sorting array, so we can't say that befor & after sorting an array all subsequences will remain the same.

=> **Subsets**

=> same as subsequences but order doesn't matter

=> only unique values

=> arr => [4 -1 2]

Subsets => { } { 4 } { -1 } { 2 } { 4, -1 } or { -1, 4 } { 4, 2 } or { 2, 4 }

or { -1, 2 } or { 2, -1 } { 4, -1, 2 } or { 2, 4, -1 } or { -1, 4, 2 } ...

sorted arr => [-1 2 4]

Subsets => { } { 4 } { 2 } { -1 } { -1, 4 } { -1, 2 } { 2, 4 } { -1, 2, 4 }

NOTE : Subsets remain same before & after sorting the given array.

* In an array of all distinct elements

no. of subsequences = no of subsets

⇒ [4 2 3]

subseq ⇒ [ ] [4] [2] [3] [4 2] [4 3] [2 3] [4 2 3]

subset ⇒ { } {4} {2} {3} {4 2} {4 3} {2 3} → { 4 2 3 }, { 3 2 4 }

ex ⇒ [6 7 7]
0 1 2

subseq ⇒ { } {6}$_0$ {7}$_1$ {7}$_2$ {6 7}$_{0,1}$ {6 7}$_{0,2}$

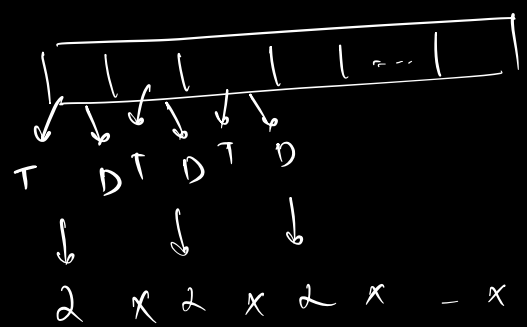
{7 7}$_{1,2}$ {6 7 7}$_{0,1,2}$

subset ⇒ { } {6}$_0$ ~~{7}$_1$~~ {7}$_2$ ~~{6 7}$_{0,1}$~~ {6 7}$_{0,2}$

{7 7}$_{1,2}$ {6 7 7}$_{0,1,2}$

⇒ { } {6} {7} {6 7} {7 7} {6 7 7}

⇒ Possible no. of subsequences :-



a_0	a_1	a_2	
x	x	x	[ ]
✓	x	x	$[a_0]$
x	✓	x	$[a_1]$
x	x	✓	$[a_2]$
✓	✓	x	$[a_0\ a_1]$
✓	x	✓	$[a_0\ a_2]$
x	✓	✓	$[a_1\ a_2]$
✓	✓	✓	$[a_0\ a_1\ a_2]$

Total no. of subsequences ⇒ 2^N

If array is distinct

no. of subsets ⇒ 2^N.

Subarray	Subsequence	Subsets
⇒ [ ] ✗	⇒ [ ] ✓	⇒ { } ✓
⇒ Cont'gous ✓	Cont'gous ✗	Cont'gous ✗
⇒ Order same as array ✓	Order same as array ✓	Order same as array ✗

	two duplicate subarrays ✓	two duplicate subsequences ✓	two duplicate subsets ✗✗
array distinct ⇒	$\frac{N(N+1)}{2}$	2^N	2^N
will remain same after sorting	not necessary	not necessary	Yes

Q:- Given an array of size N (distinct elements), check if there exists a subset with sum 'k'.

	0	1	2	3	4	5	6
arr =	3	-1	0	6	2	-3	5

k = 10

{ 5 3 2 }

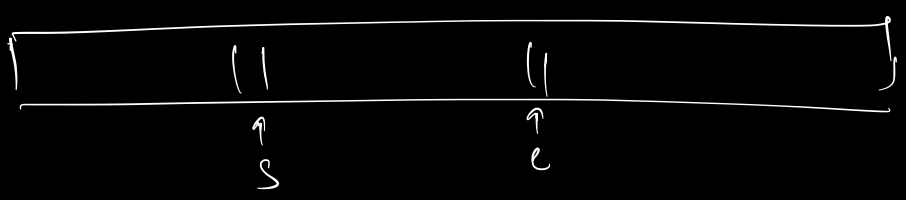
{ -1, 6, 5 }

{ 3, -1, 6, 2 }

true

k = 20

false



⇒ A ⇒ -2 6 4 → 2^3 ⇒ 8, k = 10

0 1 2

	2	1	0		Sum
0	0	0	0	→ { }	→ 0
1	0	0	1	→ {-2}	→ -2
2	0	1	0	→ {6}	→ 6
3	0	1	1	→ {-2, 6}	→ 4
4	1	0	0	→ {4}	→ 4
5	1	0	1	→ {4, -2}	→ 2
6	1	1	0	→ {4, 6}	→ 10
7	1	1	1	→ {-2, 6, 4}	→ 8

Bit Masking

generating 2^N subsets, take 0 to $2^N - 1$

↓

for each value take

elements for bit position (indexes) where bit = 1.

distinct elements

N → 2^N

↓

0 to $2^N - 1$

N bits

N = 4

$2^N \Rightarrow 2^4 \Rightarrow 16$

0 to 15 → 1 1 1 1

N = 3

$2^N \Rightarrow 2^3 \Rightarrow 8$ → 1 1 1

0 to 7

N = 2

$2^N \Rightarrow 2^2 \Rightarrow 4$ → 1 1

0 to 3

⇒ check for all subsets, if sum == k

↓

0 to $2^N - 1$

pseudo

```
for( i=0; i<2^N; i++) {
    // for i, check how many bits are equal to 1
    sum=0;
    for(j=0; j<N; j++) {
        if( checkBit(i,j) ) {
            sum = sum + arr[j]
        }
    }
    if( sum == k)
        return true
}
```

} return false

arr ⇒ 4, 3, 2 K = 6

↓

N = 3

```
for(i=0; i<2^N; i++) {
    // for i, check how many bits are equal to
    sum = 0;
    for(j=0; j<N; j++) {
        if( checkBit(i,j) ) {
            sum = sum + arr[j]
        }
    }
    if( sum == k)
        return true
}
return false
```

0 1 2

4, 3, 2 k = 6

i = 4

sum = 0

checkBit(4, 0) → –

checkBit(4, 1) ← –

checkBit(4, 2) ← 2

sum = 6 == k

return true

4 ⇒ 1 0 0

2 1 0

{arr[0], arr[2]}

TC ⇒ O(2^N * N) ⇒ O(N 2^N)

SC ⇒ O(1)

↓

TC ⇒ O(2^N) → backtracking

↓

TC ⇒ O(N * k) → DP

dynamic program

Q2. Find the sum of all subset sums.

Brute force ⇒ similar Q1, don't make sum = 0.

TC ⇒ $O(N 2^N)$

SC ⇒ $O(1)$

Optimise

arr ⇒ {-2, 6, 4} ⇒ N = 3 || 2^{N-1} ⇒ 2^3 ⇒ 2^2 ⇒ 4

{ } → 0

{-2} → -2

{6} → 6

{4} → 4

{-2, 6} → 4

{-2, 4} → 2

{6, 4} → 10

{-2, 6, 4} → 8

-2 × 4 + 6 × 4 + 4 × 4

combination technique

⇒ a_0 a_1 a_2

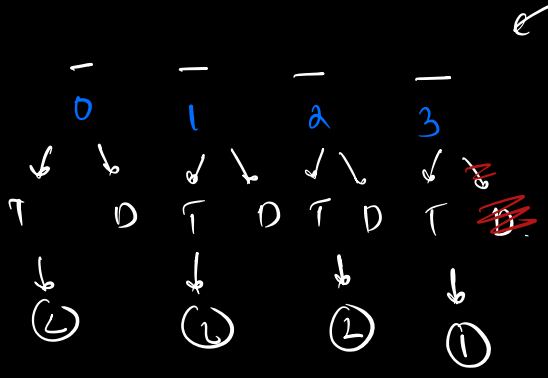
⇒ { } {a_0} {a_1} {a_2} {a_0 a_1} {a_0 a_2} {a_1 a_2} {a_0 a_1 a_2}

$0 + a_0 + a_1 + a_2 + a_0 + a_1 + a_0 + a_2 + a_1 + a_2 + a_0 + a_1 + a_2$

$= 4a_0 + 4a_1 + 4a_2 \Rightarrow 4(a_0 + a_1 + a_2)$

⇒ To find contribution of an element in total sum, we need to find. In how many subsets does the no. exist.

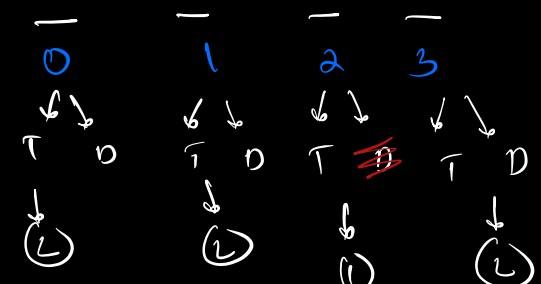
arr ⇒ [4]



⇒ 2 × 2 × 2 × 1 ⇒ 2^3

index → 2

arr [4]



→ 23

For any index i, the no. of subsets where element at it is present ⇒ 2^{N-1} [N is length of array]

pseudo

```
sum = 0
for( i = 0; i < N; i++ ){
    sum = sum + ( A[i] x 2^(N-1) )
}
return sum
```

2^{N-1} → $1 << N-1$

TC ⇒ O(N)
SC ⇒ O(1)

Leetq

Q3. Given an array of N distinct elements. Calculate (sum of all subset sums) / 2^N.

arr ⇒ a_0 a_1 a_2 a_3 ---- a_{N-1}
[N]

Sum of all subset sum ⇒ $a_0 \times 2^{N-1} + a_1 \times 2^{N-1} + a_2 \times 2^{N-1} + \cdots + a_{N-1} \times 2^{N-1}$

⇒ $2^{N-1}(a_0 + a_1 + a_2 + a_3 + \cdots + a_{N-1})$

Sum of all subset sum / 2^N ⇒ $2^{N-1}(a_0 + a_1 + a_2 + a_3 + \cdots + a_{N-1}) / 2^N$

$$\Rightarrow \frac{2^{N-1}\,(a_0 + a_1 + a_2 + \cdots + a_{N-1})}{2^{N-1} \times 2}$$

O/P $\Rightarrow$ $\frac{\text{sum of all array elements}}{2}$

TC $\Rightarrow$ O(N)

SC $\Rightarrow$ O(1)

Facebook

Q4. Given an array find the sum of max of every subsequence

A $\Rightarrow$ [3, 1, -4]

	max		max
[ ]	0	[3 -4]	3
[3]	3	[1 -4]	1
[1]	1	[3 1 -4]	3
[-4]	-4		
[3 1]	3		

Sum $\Rightarrow$ 10

Sum $\Rightarrow$ 4x3 + 2x1 + -4x1

Brute ⇒ find all subsequences

then find max & sum

TC ⇒ $O(N2^N)$

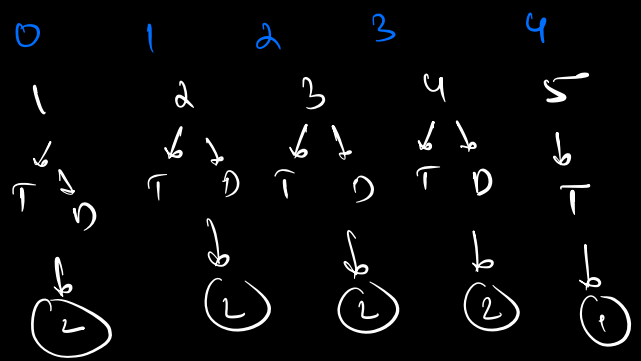
SC ⇒ $O(1)$

⇒ Contribution technique

for each element, find for how many subseq it would be max element.

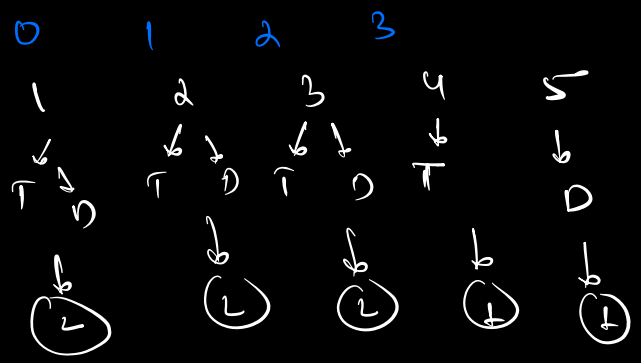
arr ⇒ [1 2 3 4 5]

how many times is 5 coming as max



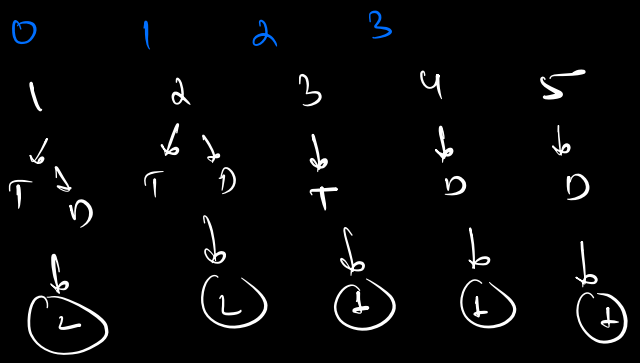
⇒ 24

how many times is 4 coming as max



$\Rightarrow 2^3$

how many times is 3 coming as max



$\Rightarrow 2^2$

pseudo

$\Rightarrow$ Sort the array

$\Rightarrow$ Contribution of each element (i)

$\Rightarrow$ $A[i] \times 2^i$

```
maxSum = 0
sort(A)
for(i=0; i<N; i++) {
    maxSum = maxSum + (A[i] x 2^i)
}
return maxSum
```

$\rightarrow$ 1<<i

TC ⇒ $O(N \log N + N) \Rightarrow O(N \log N)$

SC ⇒ — depends on sorting algo